CEFET/RJ

DAL & Comunidade

SPARKLYR

Eduardo Ogasawara
eogasawara@ieee.org
https://eic.cefet-rj.br/~eogasawara

Spark

- Fast, expressive cluster computing system compatible with Apache Hadoop
  - Works with any Hadoop-supported storage system (HDFS, S3, Avro, …)
- Improves efficiency through:
  - In-memory computing primitives
  - General computation graphs
- Improves usability through:
  - Rich APIs in Java, Scala, Python, R
  - Interactive shell

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., Stoica, I., (2016), "Apache spark: A unified engine for big data processing", Communications of the ACM, v. 59, n. 11, p. 56–65.

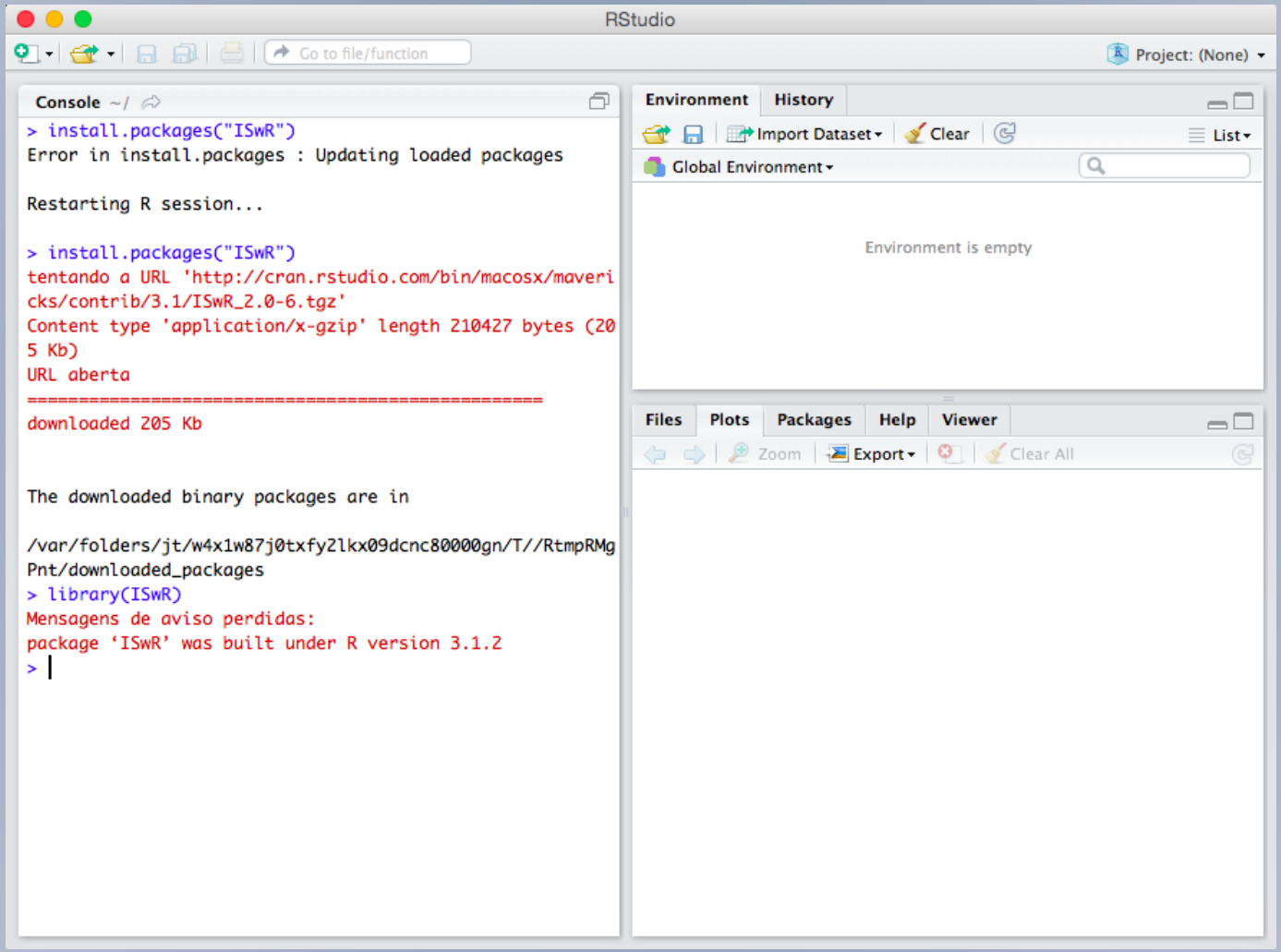
How to run it

- Local multicore: just a library in your program
- EC2: scripts for launching a Spark cluster
- Private cluster: Mesos, YARN, Standalone Mode

R

- R is a programming language and free software environment for statistical computing
  - Supported by the R Foundation for Statistical Computing
- Created by Ross Ihaka and Robert Gentleman at Auckland University, New Zealand
- R was derived by S (Bell Laboratories - AT&T)
- R is a language broadly used by statisticians, data miners, and data scientists

R Studio
http://www.rstudio.com



Great advantages: IDE with data visualization, debugging

Installing standalone

- Check if Java is installed
- Install sparklyr package

```
system('java -version')
```

```
> system('java -version')
java version "1.8.0_251"
Java(TM) SE Runtime Environment (build 1.8.0_251-b08)
Java HotSpot(TM) Client VM (build 25.251-b08, mixed mode)
[1] 0
```

```
install.packages("sparklyr")
```

```
> install.packages("sparklyr")
Installing package into 'C:/Users/eduar/OneDrive/Documents/R/win-library/3.6'
(as 'lib' is unspecified)
also installing the dependencies 'dbplyr', 'purrr'
```

```
packageVersion("sparklyr")
```

```
> packageVersion("sparklyr")
[1] '1.1.0'
```

```
library(sparklyr)
```

```
> library(sparklyr)
Warning message:
package 'sparklyr' was built under R version 3.6.3
```

Installing Spark (local multicore version)

```
spark_available_versions()
```

```
  spark
1   1.6
2   2.0
3   2.1
4   2.2
5   2.3
6   2.4
```

```
spark_install(version = "2.4")
```

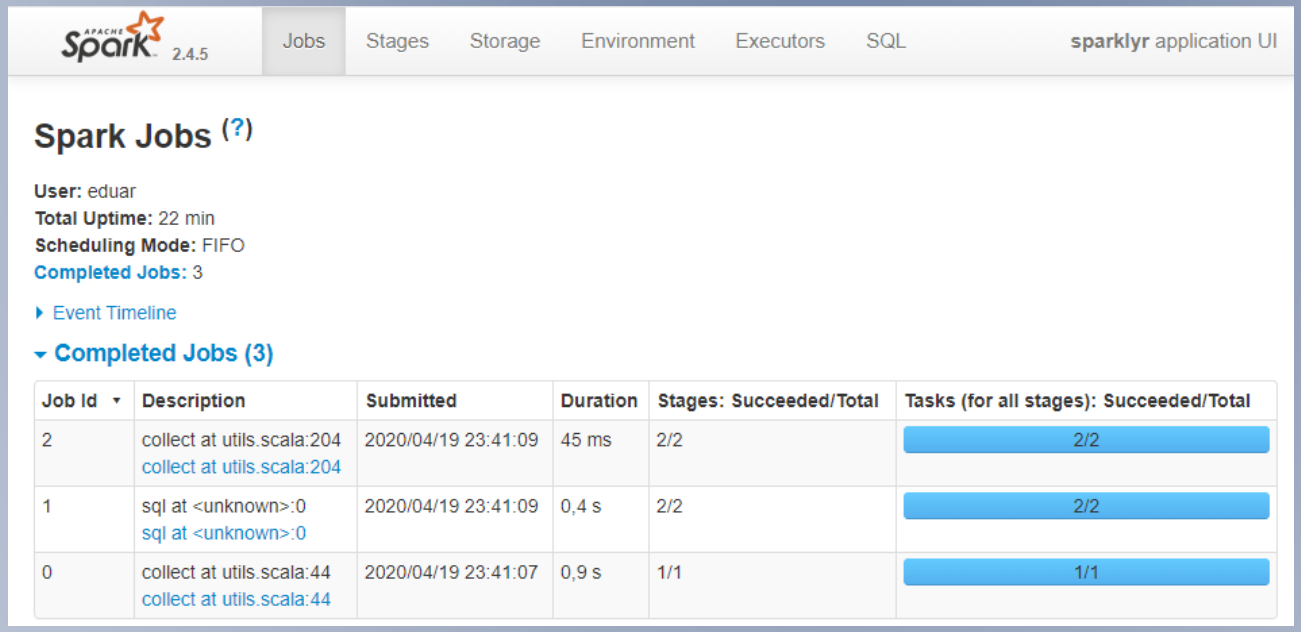
```
> spark_install(version = "2.4")
Installing Spark 2.4.5 for Hadoop 2.7 or later.
Downloading from:
- 'https://archive.apache.org/dist/spark/spark-2.4.5/spark-2.4.5-bin-hadoop2.7.tgz'
Installing to:
- 'C:\Users\eduar\AppData\Local/spark/spark-2.4.5-bin-hadoop2.7'
trying URL 'https://archive.apache.org/dist/spark/spark-2.4.5/spark-2.4.5-bin-hadoop2.7.tgz'
Content type 'application/x-gzip' length 232530699 bytes (221.8 MB)
```

```
spark_installed_versions()
```

```
> spark_installed_versions()
  spark hadoop                                                        dir
1 2.4.5    2.7 C:\\Users\\eduar\\AppData\\Local/spark/spark-2.4.5-bin-hadoop2.7
```

Connecting, staging in data, monitoring

```
library(sparklyr)
sc <- spark_connect(master = "local", version = "2.4")
data(iris)
s_iris <- copy_to(sc, iris, overwrite = TRUE)
spark_web(sc)
```



Selecting data at spark context or abstracting using dplyr

```
rm(iris)
library(DBI)
dbGetQuery(sc, "SELECT count(*) FROM iris")
```

```
> rm(iris)
> library(DBI)
> dbGetQuery(sc, "SELECT count(*) FROM iris")
  count(1)
1      150
```

```
library(dplyr)
count(s_iris)
```

```
> library(dplyr)
> count(s_iris)
# Source: spark<?> [?? x 1]
      n
  <dbl>
1   150
```

Staging out data from Spark

```
xiris <- select(s_iris, Sepal_Length, Sepal_Width,
                Petal_Length, Petal_Width, Species)  %>%
  sample_n(150) %>% group_by(Species) %>%
  summarize(Sepal_Length = mean(Sepal_Length),
            Sepal_Width = mean(Sepal_Width),
            Petal_Length = mean(Petal_Length),
            Petal_Width = mean(Petal_Width)) %>%
  collect()
```

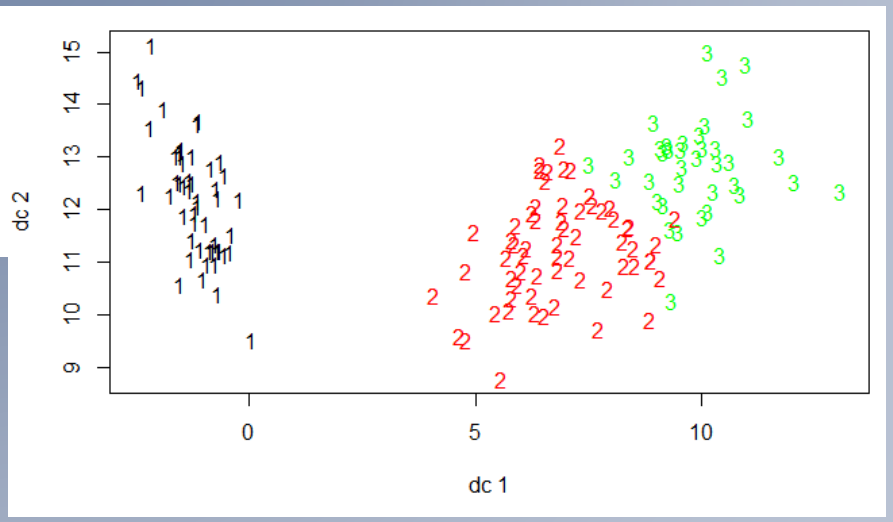
Load and save data directly in Spark

```
spark_write_csv(s_iris, "myiris.csv")

s_iris <- spark_read_csv(sc, "myiris.csv")
```

Data mining locally staging data from Spark

```
xiris <- select(s_iris, Sepal_Length, Sepal_Width,
                Petal_Length, Petal_Width)  %>%
  sample_n(150) %>% collect()

library(fpc)
clu <- kmeans(x = xiris, centers = 3)
print(clu$centers)
plotcluster(xiris, clu$cluster)
```

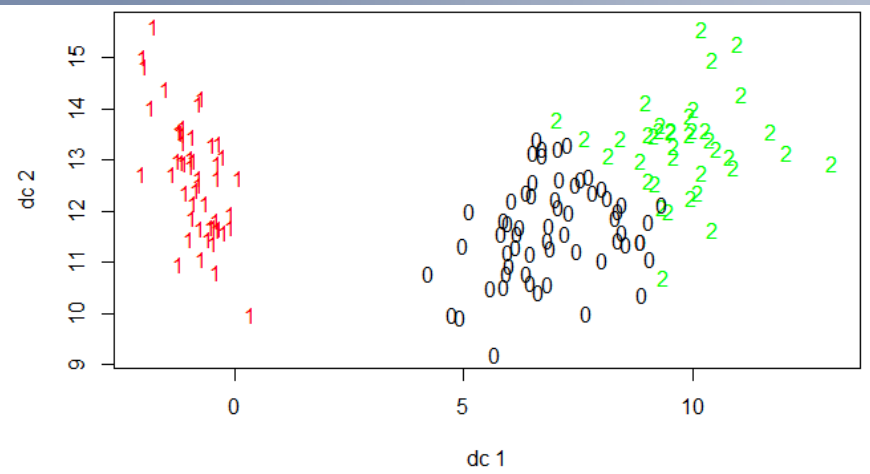
```
> library(fpc)
> clu <- kmeans(x = xiris, centers = 3)
> print(clu$centers)
  Sepal_Length Sepal_Width Petal_Length Petal_Width
1     5.006000     3.428000     1.462000    0.246000
2     5.901613     2.748387     4.393548    1.433871
3     6.850000     3.073684     5.742105    2.071053
> plotcluster(xiris, clu$cluster)
```



Data mining at Spark staging results

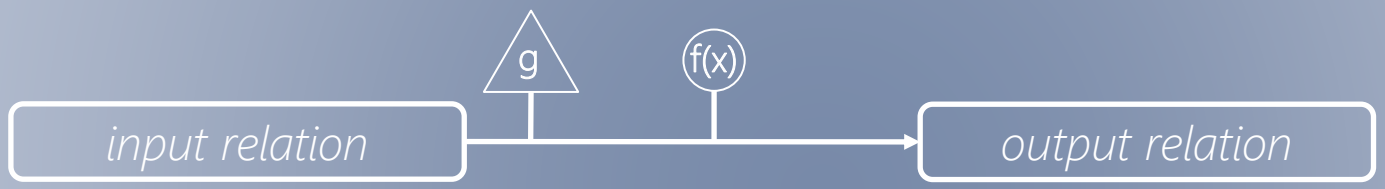
```
s_clu <- ml_kmeans(s_iris, Species ~ ., k=3)
print(s_clu)
prediction = collect(ml_predict(s_clu, s_iris))
plotcluster(xiris, prediction$prediction)
```

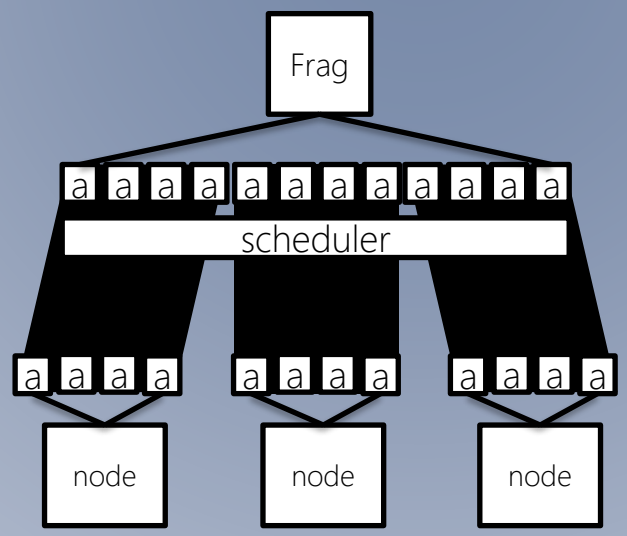
```
> plotcluster(xiris, clu$cluster)
> s_clu <- ml_kmeans(s_iris, Species ~ ., k=3)
Warning messages:
1: In spark_param_deprecated("compute_cost") :
  The 'compute_cost' parameter is deprecated in Spark 3.x
2: In spark_param_deprecated("compute_cost") :
  The 'compute_cost' parameter is deprecated in Spark 3.x
> print(s_clu)
K-means clustering with 3 clusters

Cluster centers:
  Sepal_Length Sepal_Width Petal_Length Petal_Width
1     5.883607    2.740984     4.388525    1.434426
2     5.006000    3.428000     1.462000    0.246000
3     6.853846    3.076923     5.715385    2.053846

Within Set Sum of Squared Errors =  78.85567> prediction = collect(ml_predict(s_clu, s_iris))
> plotcluster(xiris, prediction$prediction)
>
```



Distributed R

- Input relation, invoke function, output relation
- Static distribution of ***activations***





(static)

- Scenario for usage
  - Efficient R/Black-box code for handling a problem
  - Parameter exploration
  - Data fits node memory (consider the number of cores)
- spark_apply
  - x: data, f: function, group_by: partition attribute

```
f <- function(data) {
  data$id <- 1:nrow(data)
  data$max <- apply(data,1,max)
  data$min <- apply(data,1,min)
  return (data)
}

z <- s_iris %>% select(Sepal_Length, Sepal_Width,
                       Petal_Length, Petal_Width) %>%
  spark_apply(f) %>% collect
```

Map framework in R

```
library(dplyr)
library(sparklyr)

#mapping function
w_map <- function(input) {
  output <- compute(input)
  return(output)
}

#cluster connect
Sys.setenv("SPARK_HOME" = "/usr/local/spark")
sc <- spark_connect(master = "spark://rukbat:7077")

#parameter sweep
data <- data.frame(k=1:150)
data$id <- 1:nrow(data)

#distributed computing
data_out <- copy_to(sc, data, overwrite = TRUE) %>%
  spark_apply(w_map, group_by="id") %>% collect

#cluster disconnect
spark_disconnect(sc)
```

Distributed Map function

```
w_map <- function(data) {
  dataset <- NULL
  source("/rukbat/eogasawara/mycluster.R")

  for (i in 1:nrow(data)) {
    msg <- NULL
    model <- NULL

    k <- data$k[i]
    file=sprintf("/rukbat/eogasawara/test/%d.png", data$id[i])

    tryCatch(model <- func(k, file), error = function(e) {msg <<- e})

    betweenss <- 0
    if (!is.null(model)) {
      betweenss <- model$betweenss
    }
    log <- ""
    if (!is.null(msg))
      log <- msg$message

    tuple <- data.frame(k = k, betweenss=betweenss, log=log)
    dataset <- rbind(dataset, tuple)
  }
  return(dataset)
}
```

Invoked code mycluster.R

```
library(datasets)

func <- function(k, file) {
  data(iris)

 clu <- kmeans(x = iris[,1:4], centers = k)

  png(file, width = 350, height = 350)
  plotcluster(iris[,1:4], clu$cluster)
  dev.off()

  return(list(betweenss=clu$betweenss))
}
```

Monitoring the Spark cluster

rukbat.eic.cefet-rj.br/spark/

Apps Favoritos

Spark Master at spark://rukbat:7077

URL: spark://rukbat:7077
Alive Workers: 6
Cores in use: 72 Total, 72 Used
Memory in use: 102.5 GB Total, 6.0 GB Used
Applications: 1 Running, 26 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (6)

Worker Id	Address	State	Cores	Memory
worker-20200420231112-alpha-36205	alpha:36205	ALIVE	12 (12 Used)	14.5 GB (1024.0 MB Used)
worker-20200420231233-beta-34875	beta:34875	ALIVE	12 (12 Used)	14.5 GB (1024.0 MB Used)
worker-20200420231332-gamma-35527	gamma:35527	ALIVE	12 (12 Used)	14.5 GB (1024.0 MB Used)
worker-20200420231604-zeta-45245	zeta:45245	ALIVE	12 (12 Used)	30.2 GB (1024.0 MB Used)
worker-20200420231611-epsilon-35569	epsilon:35569	ALIVE	12 (12 Used)	14.5 GB (1024.0 MB Used)
worker-20200420233310-delta-33435	delta:33435	ALIVE	12 (12 Used)	14.5 GB (1024.0 MB Used)

Running Applications (1)

Application ID		Name	Cores	Memory per Executor	Submitted Time	User	State	Duration
app-20200421104405-0026	(kill)	sparklyr	72	1024.0 MB	2020/04/21 10:44:05	eogasawara	RUNNING	18 s

Monitoring the job execution

Spark 2.4.0 | Jobs | Stages | Storage | Environment | Executors | SQL | sparklyr application UI

Spark Jobs (?)

User: eogasawara
Total Uptime: 1.0 min
Scheduling Mode: FIFO
Active Jobs: 1
Completed Jobs: 4

Event Timeline

Active Jobs (1)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
4	collect at utils.scala:204 collect at utils.scala:204 (kill)	2020/04/21 10:50:12	22 s	1/2	141/201 (1 failed)

Completed Jobs (4)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
3	collect at utils.scala:204 collect at utils.scala:204	2020/04/21 10:49:41	31 s	2/2	2/2
2	collect at utils.scala:204 collect at utils.scala:204	2020/04/21 10:49:37	96 ms	2/2	2/2
1	sql at NativeMethodAccessorImpl.java:0 sql at NativeMethodAccessorImpl.java:0	2020/04/21 10:49:36	1 s	2/2	2/2
0	collect at utils.scala:44 collect at utils.scala:44	2020/04/21 10:49:34	1 s	1/1	1/1

Analyzing the results

Data	
data	150 obs. of 3 variables
data_out	149 obs. of 4 variables
sc	List of 13
Functions	
w_map	function (data)

sparklyr-example.R | data_out

Filter

	id	k	betweenss	log
93	81	81	6.788221e+02	
94	114	114	6.803164e+02	
95	33	33	6.695357e+02	
96	150	150	0.000000e+00	more cluster centers than distinct data points.
97	48	48	6.754524e+02	
98	141	141	6.811856e+02	
99	148	148	6.813606e+02	
100	97	97	6.796281e+02	
101	106	106	6.803098e+02	
102	67	67	6.775086e+02	

63	63	63	6.773959e+02	
64	64	64	6.773000e+02	
65	66	66	6.778853e+02	
66	67	67	6.775086e+02	
67	68	68	6.778954e+02	

Analyzing the files produced

```
eogasawara@rukbat:/rukbat/eogasawara/test$ ls -1 | wc -w
0
eogasawara@rukbat:/rukbat/eogasawara/test$ ls -1 | wc -w
148
```

```
eogasawara@rukbat:/rukbat/eogasawara/test$ ls -lah
total 2.6M
drwxrwxr-x 2 gpca staff  12K Apr 21 10:50 .
drwxrwxr-x 5 gpca staff 4.0K Apr 21 02:01 ..
-rw-rw-r-- 1 gpca staff  17K Apr 21 10:50 100.png
-rw-rw-r-- 1 gpca staff  16K Apr 21 10:50 101.png
-rw-rw-r-- 1 gpca staff  16K Apr 21 10:50 102.png
-rw-rw-r-- 1 gpca staff  16K Apr 21 10:50 103.png
-rw-rw-r-- 1 gpca staff  16K Apr 21 10:50 104.png
-rw-rw-r-- 1 gpca staff  15K Apr 21 10:50 105.png
-rw-rw-r-- 1 gpca staff  16K Apr 21 10:50 106.png
-rw-rw-r-- 1 gpca staff  15K Apr 21 10:50 107.png
-rw-rw-r-- 1 gpca staff  15K Apr 21 10:50 108.png
-rw-rw-r-- 1 gpca staff  15K Apr 21 10:50 109.png
```

Large Scale Example

```
Sys.setenv("SPARK_HOME" = "/usr/local/spark")
path <- "/rukbat/eogasawara/fertilizers/workflow"

parameter_sweep <- gen_cases(offset=8, replication=c(0:5), steps_ahead=c(1:8))

parameter_partition <- as.data.frame(expand.grid(steps, methods, preprocs))
colnames(parameter_partition) <- c("stepahead", "method", "preproc")
n <- nrow(parameter_partition)

for (i in 1:n) {
  method <- parameter_partition$method[i]
  preproc <- parameter_partition$preproc[i]
  stepahead <- parameter_partition$stepahead[i]
  filenameTraining <- NULL
  filenameTest <- NULL
  filenameTraining <- sprintf("%s/training_%s_%s_%d.RData", path, method, preproc, stepahead)
  filenameTest <- sprintf("%s/test_%s_%s_%d.RData", path, method, preproc, stepahead)

  if (!file.exists(filenameTraining) || !file.exists(filenameTest)) {
    parameters <- parameter_sweep %>% filter(mlearning == method & preprocessing == preproc & steps_ahead == stepahead)
    if (nrow(parameters) > 0) {
      msg <- NULL
      sc <- NULL
      out <- tryCatch(
        {
          print(sprintf("%d:%d;method=%s;preproc=%s;step_ahead=%d", i, n, method, preproc, stepahead))
          sc <- spark_connect(master = "spark://rukbat:7077") #comment this line for local debug
          dataset <- wf_apply(sc, parameters, map_train_test)
          train <- dataset %>% filter(offset != 8)
          test <- dataset %>% filter(offset == 8)
          save(train, file=filenameTraining)
          save(test, file=filenameTest)
        }
        ,
        error=function(cond) {
          msg <<- cond
        }
      )
      if (!is.null(msg)) {
        print(msg$message)
      }
      if(!is.null(sc)) {
        spark_disconnect(sc)
      }
      Sys.sleep(15)
    }
  }
}
```

Why partition the parameter space

- Too many parameters
  - Spark is not setup to consume too much memory(*)
  - Spark crashes naturally
- Independence of parameters
- Parameters can be grouped, and results can be cached…

Global Environment

Data	
parameter_partition	504 obs. of 3 variables
parameter_sweep	4786560 obs. of 11 variables

```
eogasawara@rukbat:/rukbat/eogasawara/fertilizers/workflow$ ls -lah
total 1.2G
drwxrwxr-x 2 gpca       staff 728K Apr 17 02:49 .
drwxrwxr-x 5 gpca       staff 4.0K Apr 21 14:50 ..
-rw-r--r-- 1 eogasawara staff  10K Apr 16 11:58 test_arima_dir_1.RData
-rw-r--r-- 1 eogasawara staff  11K Apr 16 12:12 test_arima_dir_2.RData
-rw-r--r-- 1 eogasawara staff  11K Apr 16 12:19 test_arima_dir_3.RData
-rw-r--r-- 1 eogasawara staff 9.8K Apr 16 12:33 test_arima_dir_4.RData
-rw-r--r-- 1 eogasawara staff  11K Apr 16 12:35 test_arima_dir_5.RData
-rw-r--r-- 1 eogasawara staff 9.6K Apr 16 09:54 test_arima_dir_6.RData
```

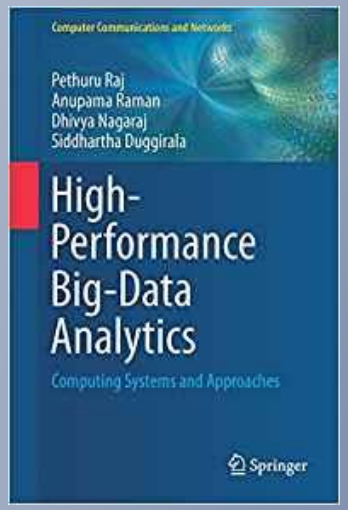
Debuging

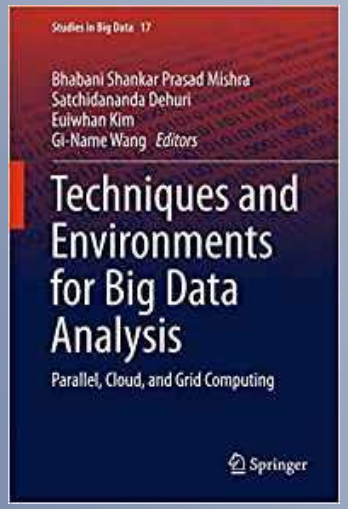
```
wf_apply <- function(sc, data, func) {
  dataset = NULL
  if (is.null(sc)) {
    for (i in 1:nrow(data)) {
      dataset <- rbind(dataset, func(data[i,]))
    }
  }
  else {
    dataset <- copy_to(sc, data, overwrite = TRUE) %>%
       spark_apply(func, group_by="id") %>% collect
  }
  return(dataset)
}
```

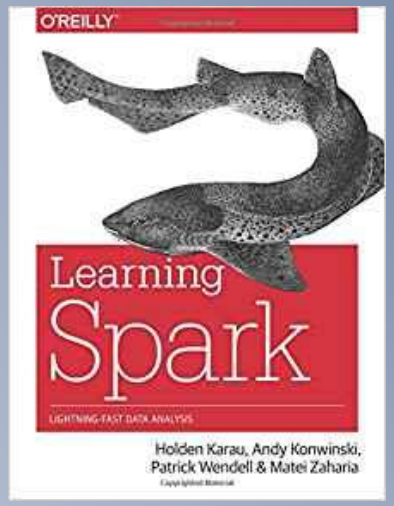
Next steps to study

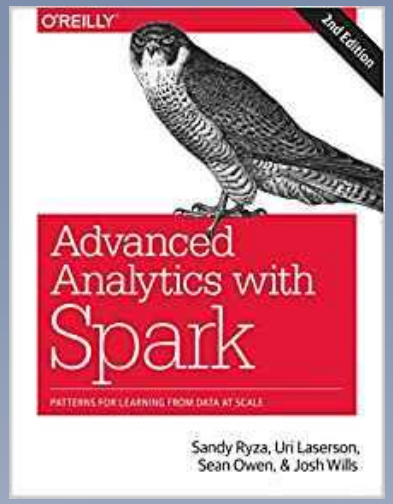
- Clusters
- Connections
- Tuning
- Extensions
- Streaming

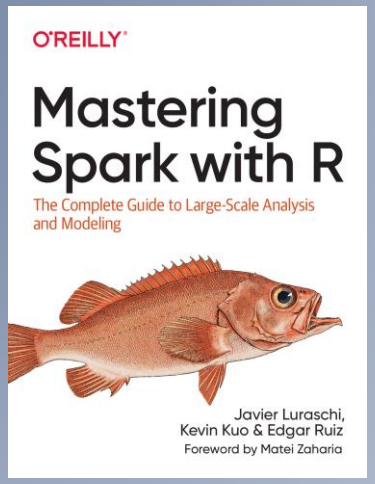
References











Further material available at https://github.com/eogasawara/R/tree/main/15-Sparkylr